PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 1: Bus Passenger Capacity Monitoring

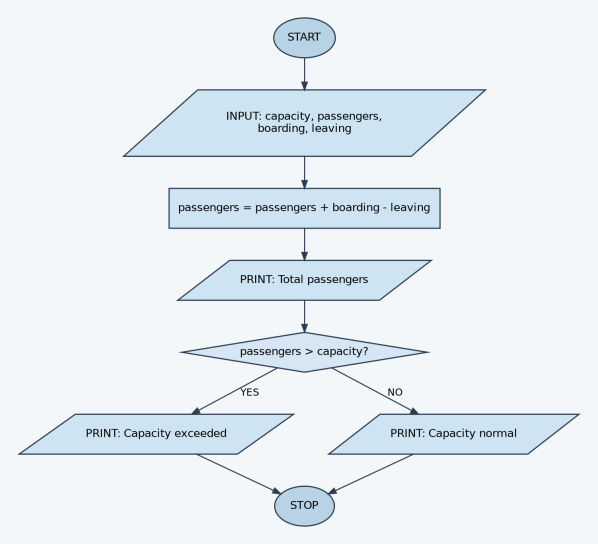
Problem Statement

Update the number of passengers on a bus after some board and some leave, then check whether the bus capacity is exceeded.

Algorithm

- Step 1: Start.
- Step 2: Read capacity, current passengers, boarding and leaving counts.
- Step 3: Compute passengers = passengers + boarding - leaving.
- Step 4: Print total passengers.
- Step 5: If passengers > capacity, print "Capacity exceeded"; else print "Capacity normal".
- Step 6: Stop.

Flowchart



Source Code & Output

```
capacity = int(input("Enter capacity: "))
passengers = int(input("Enter passengers: "))

boarding = int(input("Enter boarding: "))
leaving = int(input("Enter leaving: "))

passengers = passengers + boarding - leaving

print("Total passengers:", passengers)

if passengers > capacity:
    print("Capacity exceeded")
else:
    print("Capacity normal")
```

```
Enter capacity: 100 | Enter passengers: 56 | Enter boarding: 36 | Enter leaving: 26 | Total
passengers: 66 | Capacity normal
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 2: Sports Team Winner Decision

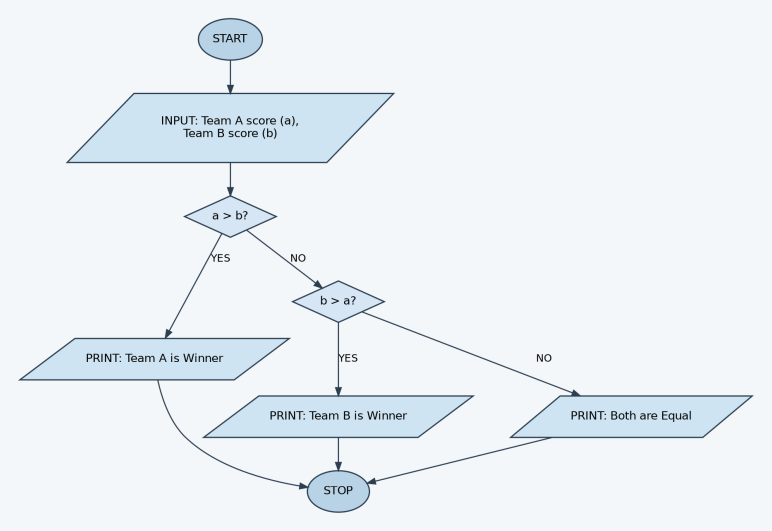
Problem Statement

Compare the scores of two teams and declare the winner, or report that both teams are equal.

Algorithm

- Step 1: Start.
- Step 2: Read Team A score (a) and Team B score (b).
- Step 3: If a > b, print "Team A is Winner".
- Step 4: Else if b > a, print "Team B is Winner".
- Step 5: Else print "Both are Equal".
- Step 6: Stop.

Flowchart



Source Code & Output

```
a = int(input("Enter Team A score: "))
b = int(input("Enter Team B score: "))

if a > b:
    print("Team A is Winner")
elif b > a:
    print("Team B is Winner")
else:
    print("Both are Equal")
```

```
Enter Team A score: 121 | Enter Team B score: 112 | Team A is Winner
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 3: Canteen Food Wastage Analysis

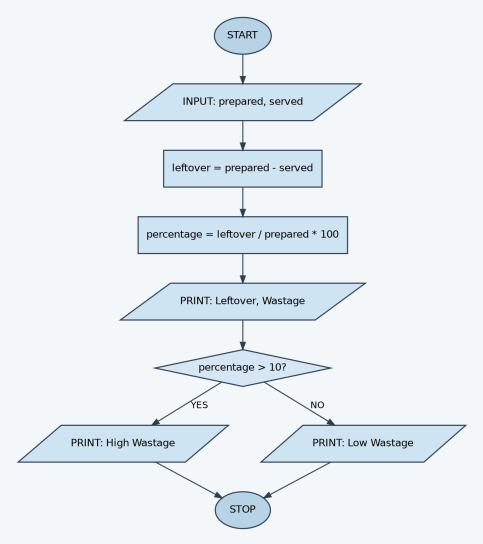
Problem Statement

Calculate the leftover meals and the wastage percentage from prepared and served meals, and classify the wastage as high or low.

Algorithm

- Step 1: Start.
- Step 2: Read prepared meals and served meals.
- Step 3: Compute leftover = prepared - served.
- Step 4: Compute percentage = (leftover / prepared) x 100.
- Step 5: Print leftover and wastage percentage.
- Step 6: If percentage > 10, print "High Wastage"; else print "Low Wastage".
- Step 7: Stop.

Flowchart



Source Code & Output

```
prepared = int(input("Enter prepared meals: "))
served = int(input("Enter served meals: "))

leftover = prepared - served

percentage = leftover / prepared * 100

print("Leftover:", leftover)
print("Wastage:", percentage)

if percentage > 10:
    print("High Wastage")
else:
    print("Low Wastage")
```

```
Enter prepared meals: 10 | Enter served meals: 5 | Leftover: 5 | Wastage: 50.0 | High Wastage
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 4: Monthly Expense Calculator

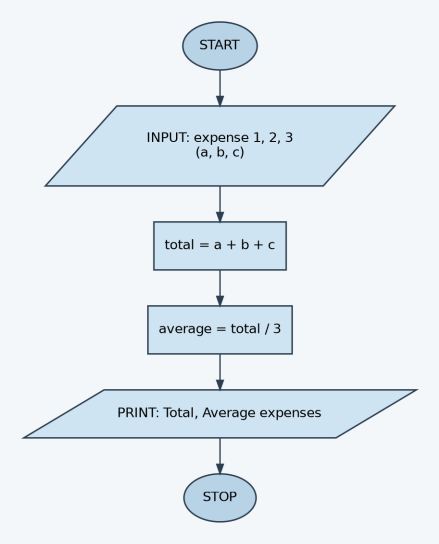
Problem Statement

Read three expense values and calculate the total and the average expense.

Algorithm

- Step 1: Start.
- Step 2: Read expense 1, expense 2 and expense 3 (a, b, c).
- Step 3: Compute total = a + b + c.
- Step 4: Compute average = total / 3.
- Step 5: Print total and average expenses.
- Step 6: Stop.

Flowchart



Source Code & Output

```
a = int(input("Enter expense 1: "))
b = int(input("Enter expense 2: "))
c = int(input("Enter expense 3: "))

total = a + b + c

average = total / 3

print("Total expenses:", total)
print("Average expenses:", average)
```

```
Enter expense 1: 12 | Enter expense 2: 15 | Enter expense 3: 20 | Total expenses: 47 | Average
expenses: 15.666666666666666
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 5: Transaction Validation using Tuples

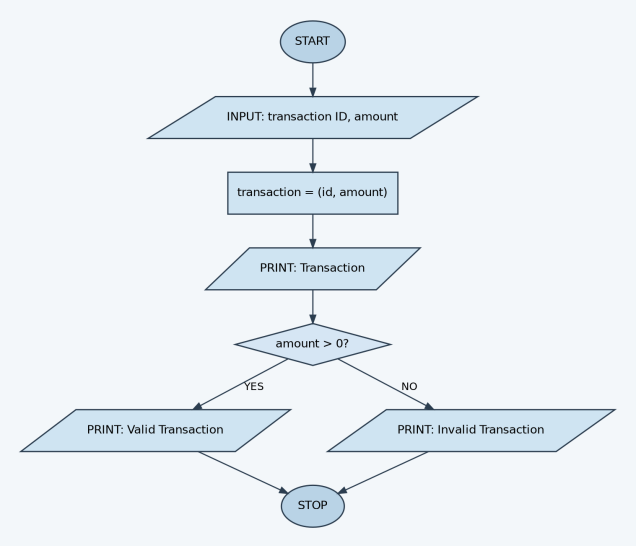
Problem Statement

Store a transaction ID and amount in a tuple, display it, and validate the transaction based on the amount.

Algorithm

- Step 1: Start.
- Step 2: Read transaction ID and amount.
- Step 3: Create tuple transaction = (id, amount) and print it.
- Step 4: If amount > 0, print "Valid Transaction".
- Step 5: Else print "Invalid Transaction".
- Step 6: Stop.

Flowchart



Source Code & Output

```
id = int(input("Enter transaction ID: "))
amount = int(input("Enter amount: "))

transaction = (id, amount)

print("Transaction:", transaction)

if amount > 0:
    print("Valid Transaction")
else:
    print("Invalid Transaction")
```

```
Enter transaction ID: 26000148 | Enter amount: 200 | Transaction: (26000148, 200) | Valid
Transaction
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 6: Exam Group Marks Calculator

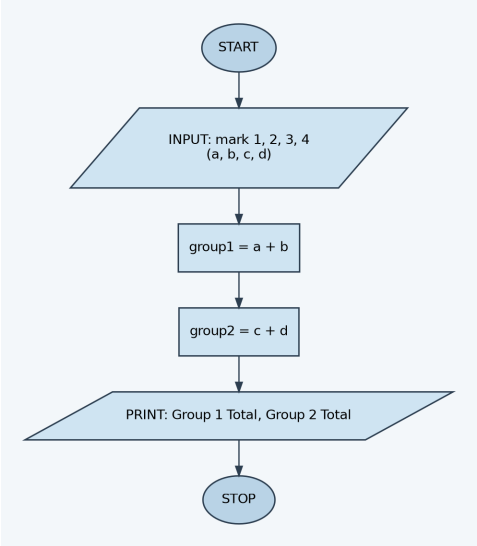
Problem Statement

Read four marks, divide them into two groups of two subjects each, and calculate each group's total.

Algorithm

- Step 1: Start.
- Step 2: Read marks a, b, c and d.
- Step 3: Compute group1 = a + b.
- Step 4: Compute group2 = c + d.
- Step 5: Print Group 1 total and Group 2 total.
- Step 6: Stop.

Flowchart



Source Code & Output

```
a = int(input("Enter mark 1: "))
b = int(input("Enter mark 2: "))
c = int(input("Enter mark 3: "))
d = int(input("Enter mark 4: "))

group1 = a + b
group2 = c + d

print("Group 1 Total:", group1)
print("Group 2 Total:", group2)
```

```
Marks: 50, 75, 54, 61 | Group 1 Total: 125 | Group 2 Total: 115
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 7: Store Billing and Stock Management

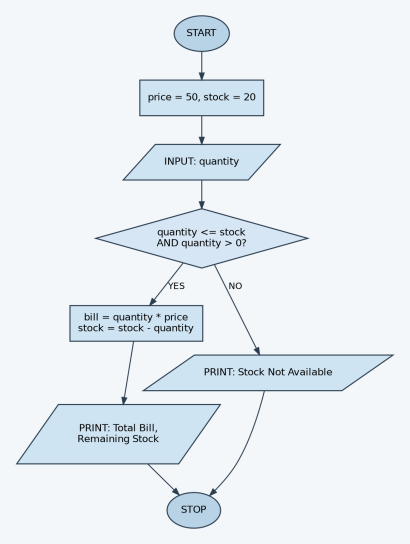
Problem Statement

Generate a bill for the quantity requested by a customer and update the remaining stock, only if enough stock is available.

Algorithm

- Step 1: Start.
- Step 2: Set price = 50 and stock = 20.
- Step 3: Read quantity.
- Step 4: If quantity <= stock and quantity > 0: bill = quantity x price, stock = stock - quantity, print bill and remaining stock.
- Step 5: Else print "Stock Not Available".
- Step 6: Stop.

Flowchart



Source Code & Output

```
price = 50
stock = 20

quantity = int(input("Enter quantity: "))

if quantity <= stock and quantity > 0:

    bill = quantity * price
    stock = stock - quantity

    print("Total Bill:", bill)
    print("Remaining Stock:", stock)

else:
    print("Stock Not Available")
```

```
Enter quantity: 42 | Stock Not Available
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 8: Library Fine Calculator

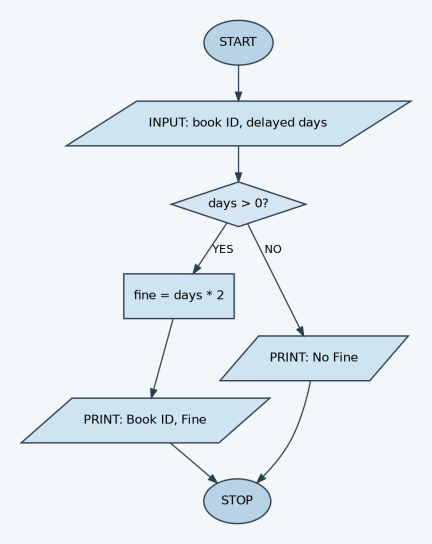
Problem Statement

Calculate the fine for a library book returned late, charging Rs. 2 per delayed day.

Algorithm

- Step 1: Start.
- Step 2: Read book ID and delayed days.
- Step 3: If days > 0, compute fine = days x 2.
- Step 4: Print book ID and fine.
- Step 5: Else print "No Fine".
- Step 6: Stop.

Flowchart



Source Code & Output

```
book = int(input("Enter book ID: "))
days = int(input("Enter delayed days: "))

if days > 0:
    fine = days * 2
    print("Book ID:", book)
    print("Fine:", fine)
else:
    print("No Fine")
```

```
Enter book ID: 26000148 | Enter delayed days: 2 | Book ID: 26000148 | Fine: 4
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 9: Bus Requirement Calculator

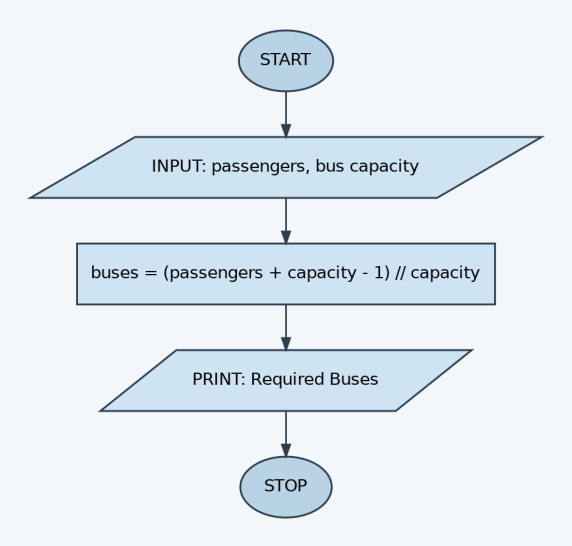
Problem Statement

Calculate the minimum number of buses required to carry all passengers, given the capacity of one bus (ceiling division).

Algorithm

- Step 1: Start.
- Step 2: Read number of passengers and bus capacity.
- Step 3: Compute buses = (passengers + capacity - 1) // capacity.
- Step 4: Print required buses.
- Step 5: Stop.

Flowchart



Source Code & Output

```
passengers = int(input("Enter passengers: "))
capacity = int(input("Enter bus capacity: "))

buses = (passengers + capacity - 1) // capacity

print("Required Buses:", buses)
```

```
Enter passengers: 112 | Enter bus capacity: 150 | Required Buses: 1
```

PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Ajay	**Roll No:** 26000148
Branch: CSE	**Course:** Python Programming

Case 10: Product Quality Defect Analysis

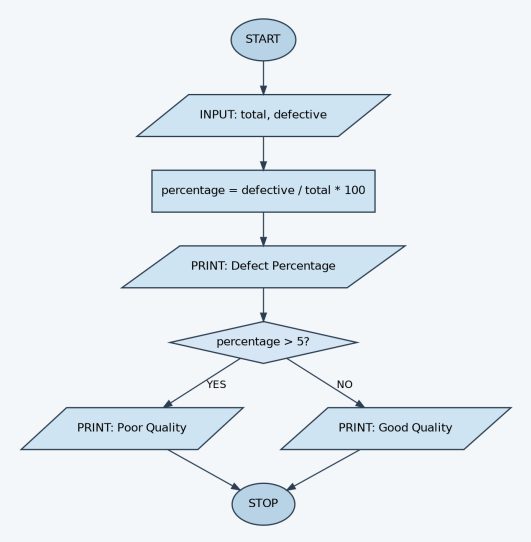
Problem Statement

Find the defect percentage from total and defective products, and classify the batch as poor or good quality.

Algorithm

- Step 1: Start.
- Step 2: Read total products and defective products.
- Step 3: Compute percentage = (defective / total) x 100.
- Step 4: Print defect percentage.
- Step 5: If percentage > 5, print "Poor Quality"; else print "Good Quality".
- Step 6: Stop.

Flowchart



Source Code & Output

```
total = int(input("Enter total products: "))
defective = int(input("Enter defective products: "))

percentage = defective / total * 100

print("Defect Percentage:", percentage)

if percentage > 5:
    print("Poor Quality")
else:
    print("Good Quality")
```

```
Enter total products: 26 | Enter defective products: 21 | Defect Percentage: 80.76923076923077 |
Poor Quality
```